# Data and the database

Sources → validation → bulk load → schema and SQL

# What you need to know

- Which exact copy of the data we use. loader/sources.json pins one revision of the upstream repository — a revision is a specific saved version, like a dated edition of a report — and stores a SHA-256 checksum for every file. A checksum is a fingerprint of the file's contents: change one character and the fingerprint changes. That is how we prove we loaded the data we documented.
- One row of our analysis is one club, in one season, in one competition. The club's name is only a label. Two spellings of the same club must never become two clubs, so every join uses the numeric club ID, not the name.
- Why a relational (SQL) database: our eleven files have fixed columns, and we constantly combine players with clubs, seasons and competitions. That combining is called a join, and relational databases are built for it. A document database would store the same facts but make these joins harder, not easier.
- What COPY does: it is PostgreSQL's bulk-loading command. It streams a whole file into a table instead of sending one INSERT statement per row, which is why 5,743,186 rows load in about ten seconds.
- Why we build indices after loading, not before. An index works like the index at the back of a book: it makes lookups fast, but it must be updated for every row inserted. Building it once after the import is faster than maintaining it during the import. ANALYZE then refreshes the statistics the database uses to pick a query plan.
- The match files from football-data.co.uk are optional. Without them the system still runs and the interface states that coverage is missing instead of showing zeros. Club names in those files are mapped to our IDs, identical fixtures load once, and contradictory scores stop the replacement before stored matches change.
- The licence of the primary source is not confirmed. We checked: there is no licence file at the pinned revision. Being publicly downloadable is not permission to reuse. Say this plainly — it is a known open item, not a mistake to hide.
- One legacy table (team_competitions_seasons) differs from the current upstream file: 196,378 rows with seven fields locally, 58,247 rows with 29 fields upstream. We kept the local file instead of silently replacing it, and the Champagne Index and the model do not use it.

# What to show

1. Open loader/sources.json on screen and read one entry aloud: the revision, the file name, its checksum and the expected column order.
2. Open sql/champagne_queries.sql and point at the GROUP BY: club ID, season, competition. Explain that MAX(team_name) only picks a label to display.
3. Open tests/test_match_loader.py and show one rejected case: a fixture whose stated result disagrees with its score never becomes a stored match.

# Your spoken part

> *Our data layer uses eleven pinned CSV tables. We chose PostgreSQL because the records have fixed schemas and are joined by player, club, competition and season. The loader validates headers, uses COPY for bulk import, and builds indices afterwards. We aggregate by canonical identifiers so name aliases do not duplicate observations. An optional match source adds actual scorelines and standings. Invalid or duplicate matches are handled explicitly. The source is publicly downloadable, but its licence is still unconfirmed; we do not claim that this requirement is resolved.*

**Hand over:** “With the data stored consistently, the next step is to define a valid season-to-season target and evaluate it.”

# Your slots on the clock

| Slide | Block | Time | Length |
|---|---|---|---|
| 1. The price of beautiful football | 01 / Question | 00:00–01:00 | 60 s |
| 2. Know the source | 02 / Data | 01:00–02:30 | 90 s |
| 3. From CSV to the screen | 03 / Architecture | 02:30–03:45 | 75 s |

# How to close the exam

1. You open the defense: slides 1–3, 00:00–03:45. You set the pace for the whole team, so speak slowly for the first twenty seconds.
2. Start with the question in plain language, then separate the two different questions immediately: what happens in the next season, and what we only observe within the same season.
3. Open loader/sources.json on the screen rather than describing it. One real manifest entry is more convincing than any slide.
4. Raise the licence yourself, before the examiner asks: the revision is pinned, the licence is not confirmed, and we do not redistribute the raw data.
5. Mention the legacy metadata table yourself too: one table differs from the current upstream version, and the central index and model do not use it.
6. Keep the database justification to one sentence: fixed columns and constant joins by player, club, season and competition.
7. Hand over with the prepared sentence instead of improvising — an improvised handover costs ten seconds you do not have.

# Where to look in the project

| Project file | What to look at in it |
|---|---|
| `loader/sources.json` | The pinned revision, file list, SHA-256 checksums and expected column order. Open this file during the defense instead of describing it. |
| `loader/get_data.py` | Header and checksum verification, download to a temporary file, atomic install. This is the answer to “what if the download is interrupted?” |
| `loader/load_data.py` | Bulk COPY into PostgreSQL, match validation, parse_match_extras, and clearing stale match rows when the optional source disappears. |
| `sql/schema.sql` | Tables, indices and additive ALTER statements for the new match columns, so an existing database upgrades without losing loaded scores. |
| `sql/champagne_queries.sql` | The aggregation key club × season × competition, and MAX(team_name) used only as a display label. |
| `sql/team_name_map.sql` | Mapping of club names to canonical IDs before any match join, so alternative spellings cannot duplicate a club. |
| `tests/test_acquisition.py` | What the acquisition tests actually prove: a wrong checksum, a wrong header and an interrupted transfer install nothing. |
| `tests/test_match_loader.py` | Duplicate and contradictory fixtures, two- and four-digit years, and valid 0–0 draws. |
| `docs/SOURCES.md` | Provenance, the legacy metadata difference and the licence boundary — your answer to the licence question. |

# Plain-language glossary

These definitions are deliberately non-technical. Use them to explain the system to someone outside the course, and to answer a question whose wording you do not recognise.

| | |
|---|---|
| **Container / Docker Compose** | A container is one packaged program with everything it needs to run. Compose is the file that starts our four containers in the right order: the database first, then the loader, then the processing pipeline, then the web application. |
| **Relational database / PostgreSQL** | A store of tables with fixed columns that can be combined by shared keys. We use PostgreSQL 16. Combining two tables on a shared key is called a join. |
| **Schema** | The declared shape of the tables: which columns exist and what type each one holds. It is written in sql/schema.sql. |
| **COPY and index** | COPY loads a whole file into a table in one stream. An index is a lookup structure, like the index of a book: it makes selective queries fast and is built after the bulk load. |

| | |
|---|---|
| **Pipeline** | The processing step between the database and the web page. It reads tables, computes our measures and the model, and writes prepared result tables back. |
| **API / endpoint / JSON** | The API is the set of web addresses the page calls. Each address is an endpoint. JSON is the plain-text format of the answer. |
| **Champagne Index** | Our own measure of attacking output: recorded goals plus assists, divided by the number of matches the source reports for that club-season. It is a proxy, not an official statistic. |
| **Target and feature** | The target is what the model tries to predict — here the change in the index in the next consecutive season. A feature is an input the model uses to predict it. |
| **Chronological holdout** | We train the model on earlier seasons and test it on later seasons it has never seen. A random split would mix the future into the training data and flatter the result. |
| **Baseline** | The simplest possible prediction, used for comparison. Ours predicts no change at all. A model is only useful if it beats the baseline on the same test rows. |
| **$R^2$ and MAE** | $R^2$ is the share of variation the model explains, from 0 to 1 — ours is 0.127, which is weak. MAE is the average absolute error in index units — ours is 0.4856 against a baseline of 0.5114, about 5% better. Neither is a percentage of correct answers. |
| **Correlation vs. causation** | A correlation (r) measures how two numbers move together, from −1 to +1. It cannot show that one causes the other. Every result in this project is a correlation, including the red-card and spending findings. |
| **Proxy** | A measurable stand-in for something we cannot measure directly. Our shots-on-target estimate is a proxy for chance quality; it is not xG, which the data source does not contain. |
| **Poisson rating** | A simple statistical model of how many goals each team scores and concedes, fitted to one league-season. It describes that season; it does not forecast new matches. |
| **SHAP** | A method that attributes a model's prediction to its individual inputs, so we can report which feature mattered most without relying on the model's internal importance score. |

# Practice

1. Explain a checksum to someone outside the course in two sentences, without using the word hash.
2. Say why loading 5.74 million rows with COPY is faster than 5.74 million INSERT statements.
3. Describe what breaks if we join clubs by name instead of by ID, and give a concrete example from the data (Inter, Internazionale).

# Questions for your part

**Why PostgreSQL rather than MongoDB?**

The source tables have fixed columns and relational links by player, club, season and competition. PostgreSQL supports those joins, native COPY and indexed lookups with a clear schema. A different database could work, but this choice follows the data shape.

**What identifies one observation?**

For the analytical output it is club ID, season and competition ID. A club's display name is a label, not its identity. We aggregate canonical keys to prevent alias names from creating duplicate observations.

**Why build indices after loading?**

Maintaining indices during a bulk load adds work to every inserted row. We import with COPY, then build the required indices and ANALYZE the updated tables so the next pipeline has current planner statistics.

**Is your source openly licensed?**

We verified the pinned source revision, headers and checksums, but did not establish an explicit upstream licence. We document this as unresolved and exclude raw data from the repository. Public downloadability alone is insufficient.

**What happens if a download is interrupted?**

The downloader writes a temporary file and installs it atomically only after checksum and header validation. An interrupted or corrupt download is not accepted as a complete source. Tests exercise these failures using fake HTTP responses.

**How do you handle duplicate match rows?**

Identical league/season/date/home/away fixtures are imported once. Conflicting scores for the same fixture stop replacement before stored match rows are changed. Missing optional input clears old enrichment on the next loader run.

**What is the main result in one sentence?**

The system is reproducible on the measured setup, while historical transfer spending alone has almost no linear relationship with next-season index change and the richer model adds only modest predictive value.

**What is the most important limitation?**

The data are retrospective and unevenly covered, the index is a proxy, and associations are not causal effects. Source licence and independent team/laptop validation also remain explicit unresolved requirements.

**How do goals for, goals against and goal difference work in the comparison?**

GF and GA are season totals rebuilt from completed home/away match scores. GD is GF − GA. They come from the optional football-data.co.uk match enrichment, not from the Champagne Index player totals.

**How are attacking and defensive ranks calculated?**

Within one league and season, attack rank orders goals scored per match from highest to lowest; defense rank orders goals conceded per match from lowest to highest. SQL RANK gives tied rates the same rank and leaves a gap after ties. They describe scores, not tactics or chance quality.

**Do we have xG or can we estimate it?**

The source has no provider xG field. The comparison instead shows observed shots and shots on target. A separate estimate multiplies shots on target by the league-season average goals-per-shot-on-target rate; it is an experimental volume proxy with no shot-location or chance-quality input, not xG.

**What is the difference between the Champagne Index and attack rank?**

The Champagne Index is recorded player goals plus assists divided by a Transfermarkt matches proxy for a club-season. Attack rank uses actual completed league scorelines, sums team goals and ranks goals per match within the league-season. Their sources, coverage and units differ.

**How did you check the goalkeeper defensive totals?**

We compared Transfermarkt goalkeeper goals-conceded totals with the independent match-result totals on 388 adequately covered top-five club-seasons. They match exactly in 366 (94.3%). Clean sheets match only 269 (69.3%), so the scoreline source remains the basis of league ranks.

**What does the defensive-spend analysis show?**

The Pearson correlation between defender transfer fees and next-season change in the goalkeeper-source conceded-goal rate is r=0.004047, p=0.426997, n=38,531. The sample does not support a useful linear association. It is descriptive, not causal. A separate match-level check finds teams with a red card concede 0.341 more goals on average in the same match, but match context confounds that association.

**Why keep the player-source defensive values separate from scoreboard goals?**

The sources use different records and denominators. Goalkeeper conceded totals agree with scorelines in 94.3% of adequately covered seasons, while clean sheets agree in 69.3%. The UI therefore uses match scores for GF, GA, GD and ranks; player-source totals remain a quality cross-check. Shots and corners are also observed match statistics, not chance-quality measures.

**Is the shot-based estimate the same as xG?**

No. It multiplies a team’s shots on target by the league-season average goals per shot on target. It has no shot-location, distance, body-part, pressure or goalkeeper-position input. It is a rough volume benchmark and must be called an SOT-based proxy, not provider xG.

**What does the Poisson attack/defense rank tell us?**

The model fits attack, defense and a shared home-advantage term to the selected league-season scores. Larger fitted attack and defense parameters mean stronger estimated attack and defense. Its ranks are descriptive and in-sample; the separate chronological RandomForest evaluation is the project’s held-out prediction check.

**Did the added defensive features improve the model?**

On the same chronological split, $R^2$ moves from 0.1271 with the five-feature baseline to 0.1502 after adding goalkeeper conceded rate, discipline and net spend; MAE moves from 0.4856 to 0.4788 on 5,585 held-out rows. The gain is modest, and it does not isolate a causal effect of defense.

**Do red cards cause more goals to be conceded?**

The local sample shows a same-match difference of +0.341 conceded goals per team-match (1,619 sent-off vs 16,195 others; Welch $p < 0.0001$). This is observational: stronger opponents, match state and timing can affect both red cards and goals. It is not a causal estimate.

# Phrasing to avoid

- Our data is open source.
- The database cleans the data for us.
- A zero transfer fee means the player moved for free.

# Exam day

| Date | Format | What is required |
|---|---|---|
| **12 November 2026**<br>CIDS Lecture 15, pp. 37–39 | Checkpoint · 5 minutes | One end-to-end demo, one visible result and the first measured full-run time. A polished interface is not required. |
| **20 / 26 November 2026**<br>CIDS Lecture 15, pp. 37–39 | Final defense · 15 minutes + 5 minutes of questions | Every real participant speaks. Suggested split: question and data 3 min, architecture 2, live demo 3, measurements 3, problems 2, contributions 2. |

## Machine and materials

1. Thirty minutes ahead: run `docker compose up -d --build`, then open /api/health and wait for `ready=true` with the loader and pipeline finished.
2. Open four tabs in advance: Overview (Serie A 24/25), the Inter / Milan comparison, the Lab, and the Project page with the numbers.
3. Open the offline study site `defense/site/index.html` and `defense/site/downloads/Presentation.pdf`: both work without a network and without the database.
4. Check that the screenshots in `defense/site/assets/*-desktop.png` are present. They are the fallback if Docker does not come up.
5. Turn off notifications, connect the charger, test screen mirroring and increase the browser font size.
6. Decide in advance who drives the demo and who watches the clock. The timer is built into `defense/site/presentation.html` (N for notes, F for fullscreen).

## How to answer questions

- Answer in three steps: one short sentence, then the evidence (a file or a number), then the limitation.
- Give every number with its sample and unit: “r = −0.79 across 388 club-seasons, within the same season”, not “the correlation is high”.
- If you do not know, say where the answer lives instead of guessing: “that is in docs/MEASUREMENTS.md, the memory sampling method is described there”.
- If the question belongs to someone else's part, hand it over in one sentence rather than approximating their area.
- Never turn an association into a cause. Say “accompanies” or “is associated with”, never “causes” or “improves”.
- If asked about a weak point, answer it first and directly. An acknowledged limitation is stronger than a defended inaccuracy.

## Uncomfortable questions: answer them first

| Topic | Question | Answer and evidence |
|---|---|---|
| Licence of the primary source | Is your data openly licensed? | We pinned the source revision and verified headers and checksums, but there is no licence file, so we did not establish an explicit reuse licence. We document it as unresolved and we do not redistribute the raw data. Public downloadability is not a licence. `docs/SOURCES.md, docs/LICENCE_DECISION.md` |
| Four contributors | Who did what in this project? | Describe only the work you can open and explain on screen. Git identities and role cards are not proof of independent contribution, and the repository says so. `docs/WORK_PACKAGES.md, README.md` |
| Reproduction on another machine | Did someone else reproduce it? | The run is reproducible on the measured machine, and we also rebuilt it from a clean clone. An independent fresh checkout on another person's laptop has not been recorded yet, so we do not claim it. `docs/FRESH_CHECKOUT.md, docs/MEASUREMENTS.md` |
| xG and proxies | Is this xG? | No. The source has no provider xG field. We show observed shots and shots on target, and a shots-on-target conversion proxy that we label experimental. The Poisson attack and defense ratings describe the season they are fitted on. `docs/GLOSSARY.md, pipeline/proxy_xg.py, pipeline/power_ratings.py` |

> A role is a study responsibility. Do not claim authorship or a verification you did not perform. Show exactly the changes and commands you ran yourself.